Lab2 Report

Name & Date

黄思睿 PB23151782　　2024/10/25

Purpose

本实验的目的是使用 LC-3 汇编语言实现对柯拉兹猜想的验证。柯拉兹猜想提出，对于任意自然数 a_0，其序列经过以下规则的变换，最终都会收敛到 1：

$$a_n = \begin{cases} \frac{a_{n-1}}{2} & \text{如果 } a_{n-1} \text{ 是偶数} \\ 3a_{n-1} + 1 & \text{如果 } a_{n-1} \text{ 是奇数} \end{cases} \tag{1}$$

我们将实现一个程序，该程序从内存位置 `x3100` 中读取一个初始值a_0，然后计算出该序列到达 1 所需的步数，并将步数存储在内存`x3101` 中。

The core algorithm & Critical code

步骤 1：加载初始值

- 使用 `LDI` 指令将初始值 a_0 从内存位置 `x3100`加载到寄存器 R0 中。如果 a_0已经是 1，程序将立即终止。

```
START   LDI R0, NUM          ; 从内存 x3100 中加载初始值 a0 到寄存器 R0
        BRz END
```

步骤 2：步数计数器初始化

- 使用 `AND` 指令将 R2 清空，R2 用作步数计数器，每次循环步数增加 1。

```
 AND R2, R2, #0       ; 清空 R2，R2 用来计数 k 的值（步数）
```

步骤 3：判断奇偶性

- 通过 `AND R3, R0, #1` 检查当前数字是奇数还是偶数。如果是偶数，跳转到处理偶数的分支。

```
LOOP    ADD R2, R2, #1       ; 每次循环增加步数计数器 k
        AND R3, R0, #1       ; 检查当前数字是偶数还是奇数
        BRz EVEN             ; 如果 R3 是 0，则数字为偶数，跳转到 EVEN
```

步骤 4：奇数处理

- 当 a_0 为奇数时，使用 `ADD` 指令执行 $3*a_0+1$。这个过程需要通过两次加法实现 ，并且在完成后更新 a_0的值。

```
; 奇数情况：计算 3 * a0 + 1
        AND R4, R4, #0
        ADD R4, R0, R0        ; R4 寄存 2 * a0
        ADD R0, R4, R0        ; 更新 R0 变为 3 * a0
        ADD R0, R0, #1        ; 加 1，得到 3 * a0 + 1
        BRnzp CHECK1           ; 跳转到检查 a0 是否等于 1
```

步骤 5：偶数处理

- 当 a_0 为偶数时，我们希望将 a_0 除以 2。

  由于 LC-3 没有直接的除法指令，我们通过从 0 向上遍历直到找到 a_0 /2 来实现除以 2 的效果。

```
EVEN    ; 偶数情况：a0 = a0 / 2
        AND R4, R4, #0        ; R4  初始化为 0

UP      ADD R4, R4, #1        ; R4 自增
        BRnzp CHECK2

UPDATE  ADD R0, R4, #0        ; 更新 R0 的值


CHECK2  ; 检查 R4 是否为满足目标的 R2/2
        AND R5, R5, #0
        ADD R5, R4, #0
        ADD R5, R5, R5       ; R5 = 2* R5
        NOT R5, R5           ; R5 = NOT R5
        ADD R5, R5, #1       ; R5 = R5 + 1（取反加一）;
        ADD R5, R0, R5       ; R0 和 2* R4 相减法， 结果存储在 R5
        BRnp UP              ; 如果结果 非0， R4 继续自增
        BRz UPDATE           ;
```

步骤 6：循环和跳转

- 程序进入循环，重复上述判断，直到 a_0 变为 1。每次循环结束后，将步数增加 1。

```
CHECK1   ; 检查当前 a0 是否为 1
        ADD R3, R0, #-1       ; 将当前值减去 1
        BRz STORE_RESULT      ; 如果 R3 结果为 0，跳转到存储结果
        BRnzp LOOP            ; 否则继续循环
```

步骤 7：存储结果

- 当 a_0 到达 1 时，将步数存储到内存位置 `x3101` 中。使用 `STI` 指令将步数 k 存储到指定内存地址。

```
STORE_RESULT
        STI R2, K_COUNT       ; 将步数计数器 k 存储到内存 x3101 中
```

全部汇编代码：

```
        .ORIG x3000          ; 程序从地址 x3000 开始

START   LDI R0, NUM           ; 从内存 x3100 中加载初始值 a0 到寄存器 R0
        BRz END
        AND R2, R2, #0       ; 清空 R2，R2 用来计数 k 的值（步数）


LOOP    ADD R2, R2, #1       ; 每次循环增加步数计数器 k
        AND R3, R0, #1       ; 检查当前数字是偶数还是奇数
        BRz EVEN             ; 如果 R3 是 0，则数字为偶数，跳转到 EVEN

        ; 奇数情况：计算 3 * a0 + 1
        AND R4, R4, #0
        ADD R4, R0, R0       ; R4 寄存 2 * a0
        ADD R0, R4, R0       ; 更新 R0 变为 3 * a0
        ADD R0, R0, #1       ; 加 1，得到 3 * a0 + 1
        BRnzp CHECK1          ; 跳转到检查 a0 是否等于 1

EVEN    ; 偶数情况：a0 = a0 / 2
        AND R4, R4, #0       ; R4  初始化为 0

UP      ADD R4, R4, #1       ; R4 自增
        BRnzp CHECK2

UPDATE  ADD R0, R4, #0       ; 更新 R0 的值

CHECK1   ; 检查当前 a0 是否为 1
        ADD R3, R0, #-1      ; 将当前值减去 1
        BRz STORE_RESULT     ; 如果 R3 结果为 0，跳转到存储结果
        BRnzp LOOP           ; 否则继续循环

CHECK2  ; 检查 R4 是否为满足目标的 R2/2
        AND R5, R5, #0
        ADD R5, R4, #0
        ADD R5, R5, R5      ; R5 = 2* R5
        NOT R5, R5          ; R5 = NOT R5
        ADD R5, R5, #1      ; R5 = R5 + 1（取反加一）;
        ADD R5, R0, R5      ; R0 和 2* R4 相减法， 结果存储在 R5
        BRnp UP             ; 如果结果 非0， R4 继续自增
        BRz UPDATE          ;
```

```
STORE_RESULT
        STI R2, K_COUNT       ; 将步数计数器 k 存储到内存 x3101 中

END     TRAP x25             ; 程序结束

; DATA DECLARATIONS          ; 数据声明

NUM     .FILL x3100          ; a0 的初始值存储在内存地址 x3100
K_COUNT .FILL x3101          ; k 的结果将存储在内存地址 x3101

        .END                 ; 程序结束
```

Debug process

问题 & 问题解决：

- **奇偶判断问题**：
  - 在最初碰到这个问题时，不知道怎么实现奇偶判断，后来想到可以根据数字的末尾（0 或 1）来和 1 进行 `AND` 运算实现。
  - 在最初实现中，我们曾误用 `BRnp` 指令，它会忽略某些条件下的分支跳转。在调试过程中，我们将其替换为 `BRnzp`，以保证任何条件下都能跳转。
- **模拟除法**：
  - LC-3 不支持直接除法。我们通过将`R4` 从 0 开始递增，将`2*R4`与`R0`进行比较，直至`R4 == R0/2`，从而得到 `R0/2`。
- **代码的测试问题**：
  - 由于每次运行代码前我都会把机器重新初始化，所以`x3100`的位置存储的数据一直为 0 ，不利于代码可行性的检验。所以我们一开始编写这个程序时，在程序顶端再多写了几行内容，用于先向`x3100`中写入数据，再进行我们的程序，具体如下：

    ```
    LD  R6, My_Val        ; 从 My_Val中读取数据，存入到R6中
    STI R6, NUM           ; 将 R6 数据存储到 NUM 的地址中

    ...                   ; 我们的程序

    ; DATA DECLAIRATION 中加入
    My_Val  .FILL #16     ; 数据可为任意你想要测试的样例
    ```

- **数据的存储和读写问题**：
  - 一开始并不了解 LD 系列操作和 ST 系列操作的用法，后来上网查找以及翻看YouTube上的视频才了解，并学会使用。

Run Results

测试样例和测试结果：

- a_0 = R0 = 16

> R0 初始化 为16
>
> **Registers**
>
> | | | | |
> |---|---|---|---|
> | R0 | x0010 | 16 | |
> | R1 | x0000 | 0 | |
> | R2 | x0000 | 0 | |
> | R3 | x0000 | 0 | |
> | R4 | x0000 | 0 | |
> | R5 | x0000 | 0 | |
> | R6 | x0010 | 16 | |
> | R7 | x0000 | 0 | |
> | PSR | x8001 | 32769 | CC: P |
> | PC | x3003 | 12291 | |
> | MCR | x0000 | 0 | |
>
> **Console (click to focus)**
>
> **Memory**
>
> | | | | |
> |---|---|---|---|
> | x3000 | x2C1F | 11295 | *LD R6, My_Val* |
> | x3001 | xBC1C | 48156 | *STI R6, NUM* |
> | x3002 | xA01B | 40987 | *START LDI R0, NUM* |
> | x3003 | x0419 | 1049 | *BRz END* |
> | x3004 | x54A0 | 21664 | *AND R2, R2, #0* |
> | x3005 | x14A1 | 5281 | *LOOP ADD R2, R2, #1* |
> | x3006 | x5621 | 22049 | *AND R3, R0, #1* |
> | x3007 | x0405 | 1029 | *BRz EVEN* |
> | x3008 | x5920 | 22816 | *AND R4, R4, #0* |
> | x3009 | x1800 | 6144 | *ADD R4, R0, R0* |
> | x300A | x1100 | 4352 | *ADD R0, R4, R0* |
> | x300B | x1021 | 4129 | *ADD R0, R0, #1* |
> | x300C | x0E04 | 3588 | *BRnzp CHECK1* |
>
> `x3100` 存储的是 16
>
> **Registers**
>
> | | | | |
> |---|---|---|---|
> | R0 | x0010 | 16 | |
> | R1 | x0000 | 0 | |
> | R2 | x0000 | 0 | |
> | R3 | x0000 | 0 | |
> | R4 | x0000 | 0 | |
> | R5 | x0000 | 0 | |
> | R6 | x0010 | 16 | |
> | R7 | x0000 | 0 | |
> | PSR | x8001 | 32769 | CC: P |
> | PC | x3003 | 12291 | |
> | MCR | x0000 | 0 | |
>
> **Memory**
>
> | | | |
> |---|---|---|
> | x3100 | x0010 | 16 |
> | x3101 | x0000 | 0 |
> | x3102 | x0000 | 0 |
> | x3103 | x0000 | 0 |
> | x3104 | x0000 | 0 |
> | x3105 | x0000 | 0 |
> | x3106 | x0000 | 0 |
> | x3107 | x0000 | 0 |
> | x3108 | x0000 | 0 |
> | x3109 | x0000 | 0 |
> | x310A | x0000 | 0 |
>
> 程序运行完毕， `x3101` 存储的是 4， 符合预期

> **Registers**

R0	x0000	0	
R1	x7FFF	32767	
R2	x0004	4	
R3	x0000	0	
R4	x0001	1	
R5	x0000	0	
R6	x2FFE	12286	
R7	x0000	0	
PSR	x0002	2	CC: Z
PC	x036C	876	
MCR	x0000	0	

> **Memory**

x3100	x0010	16
x3101	x0004	4
x3102	x0000	0
x3103	x0000	0
x3104	x0000	0
x3105	x0000	0
x3106	x0000	0
x3107	x0000	0
x3108	x0000	0
x3109	x0000	0
x310A	x0000	0

- **a_0 = R0 = 96**

> R0 初始化为 96， `x3100` 存储的是 96

> **Registers**

R0	x0060	96	
R1	x0000	0	
R2	x0000	0	
R3	x0000	0	
R4	x0000	0	
R5	x0000	0	
R6	x0060	96	
R7	x0000	0	
PSR	x8001	32769	CC: P
PC	x3003	12291	
MCR	x0000	0	

> **Memory**

x3100	x0060	96
x3101	x0000	0
x3102	x0000	0
x3103	x0000	0
x3104	x0000	0
x3105	x0000	0
x3106	x0000	0
x3107	x0000	0
x3108	x0000	0
x3109	x0000	0
x310A	x0000	0

> 程序运行完毕， `x3101` 存储的是 12， 符合预期

> **Registers**

R0	x0000	0	
R1	x7FFF	32767	
R2	x000C	12	
R3	x0000	0	
R4	x0001	1	
R5	x0000	0	
R6	x2FFE	12286	
R7	x0000	0	
PSR	x0002	2	CC: Z
PC	x036C	876	
MCR	x0000	0	

> **Memory**

x3100	x0060	96
x3101	x000C	12
x3102	x0000	0
x3103	x0000	0
x3104	x0000	0
x3105	x0000	0
x3106	x0000	0
x3107	x0000	0
x3108	x0000	0
x3109	x0000	0
x310A	x0000	0

- **a_0 = R0 = 7**

> R0 初始化为 96， `x3100` 存储的是 7

Registers			
R0	x0007	7	
R1	x0000	0	
R2	x0000	0	
R3	x0000	0	
R4	x0000	0	
R5	x0000	0	
R6	x0007	7	
R7	x0000	0	
PSR	x8001	32769	CC: P
PC	x3003	12291	
MCR	x0000	0	

Memory		
x3100	x0007	7
x3101	x0000	0
x3102	x0000	0
x3103	x0000	0
x3104	x0000	0
x3105	x0000	0
x3106	x0000	0
x3107	x0000	0
x3108	x0000	0
x3109	x0000	0
x310A	x0000	0

程序运行完毕，`x3101`存储的是 16， 符合预期

Registers			
R0	x0000	0	
R1	x7FFF	32767	
R2	x0010	16	
R3	x0000	0	
R4	x0001	1	
R5	x0000	0	
R6	x2FFE	12286	
R7	x0000	0	
PSR	x0002	2	CC: Z
PC	x036C	876	
MCR	x0000	0	

Memory		
x3100	x0007	7
x3101	x0010	16
x3102	x0000	0
x3103	x0000	0
x3104	x0000	0
x3105	x0000	0
x3106	x0000	0
x3107	x0000	0
x3108	x0000	0
x3109	x0000	0
x310A	x0000	0

Conclusion：

通过这些测试，我确认程序正确地执行了 Collatz 猜想的运算，并准确计算了每个输入值到达 1 所需的步数。

Suggestions

1. **优化代码结构：**
   当前代码中对于除法的模拟操作较为繁琐。未来说不定可以进一步简化循环逻辑，使得程序运行效率更高。
2. **增加更多测试样例：**
   虽然目前程序已通过多个测试，但可以考虑增加极端情况的测试，比如较大的 a0a_0a0 值，以验证程序的鲁棒性。